Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting appropriate generative AI models for the ComicCraft application. This involves researching the capabilities and performance of multiple AI models to ensure that the chosen models align well with the application's objectives of generating comic storylines, character dialogues, scene descriptions, and comic-style illustrations.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

Review the specific needs of the ComicCraft application, focusing on the types of content it must generate:

- Structured comic outlines (panel-by-panel)
- Full comic narration and character dialogues
- Visual comic-style illustrations for each panel

The system must deliver creative, engaging, and visually rich comic outputs for users based on their custom story prompts.

Model Evaluation

Research documentation and capabilities of:

- **Google's Gemini Models**: Gemini Flash (optimized for fast structured outputs) and Gemini Pro (optimized for detailed, creative text generation).
- **Hugging Face Diffusers**: Stable Diffusion model (for high-quality AI-generated images based on prompts).

Compare the selected models based on:

- Response quality for creative writing tasks (narration and dialogues)
- Prompt adherence and richness of detail for scene descriptions
- Speed and resolution of image generation
- Model stability, available documentation, and community support

Practical testing was conducted using different prompts to assess each model's storytelling abilities and image generation performance.

Final Model Selection

Choose the AI models that best match ComicCraft's project needs:

- **Gemini Flash (models/gemini-1.5-flash)**:
  Selected for fast and structured generation of comic panel outlines based on story prompts.
- **Gemini Pro (models/gemini-1.5-pro)**:
  Selected for detailed, creative comic narration and engaging character dialogues.
- **Stable Diffusion (runwayml/stable-diffusion-v1-5)**:
  Selected for generating high-quality comic-style illustrations from image prompts.

This combination ensures that ComicCraft can deliver rich, creative storytelling alongside vivid, visually appealing comic panels in an automated, efficient manner.

Activity 1.2.Define the architecture of the application

Architecture Overview: The architecture of ComicCraft is structured into three primary components:

- **Frontend** (HTML, CSS, Jinja2)
- **Backend** (FastAPI application)
- **AI Integration** (Google Gemini APIs and Hugging Face Diffusers)

The frontend captures user inputs and displays generated comics, the backend handles data processing and routing, and the AI services generate story content and illustrations.
Below is a high-level description of each component's role in the system.

Frontend Responsibilities:

- Users interact with a simple and user-friendly web interface developed using **HTML, CSS, and Jinja2 templates**.
- Key user input fields include:
  - **Story Prompt**: Main idea for the comic
  - **Character Name**: Hero of the comic
  - **Setting**: Location (e.g., forest, school, city)
  - **Story Tone**: Mood of the story (e.g., dramatic, funny)
  - **Art Style**: Visual style preference (e.g., anime, realistic)
- Upon form submission, data is sent to the FastAPI backend via a POST request for processing.

Templates Used:

- index.html: For user input collection
- comic_preview.html: To display the generated comic
- export_success.html: To show a success message after comic export

The frontend ensures a seamless flow from input collection to comic generation display.

Backend Responsibilities:

- **FastAPI** is used to manage server-side operations and route handling.
- Key backend tasks:
  - Receive form data from the frontend or JSON API calls.
  - Call AI models to generate panel outlines, story narration, and illustrations.
  - Organize and build the complete comic layout.
  - Export the comic to a downloadable PDF format.
- Routes are defined in routes.py to handle:
  - Form-based and JSON-based comic generation
  - Image testing
  - Comic export confirmation

The backend ensures proper coordination between user inputs, AI generation workflows, and template rendering.

AI Integration Points:

- **Gemini Flash** is called to generate the structured 5-panel comic outline based on the user's story prompt.
- **Gemini Pro** is called next to create detailed narration and character dialogue from the outline.
- **Stable Diffusion** is used to generate comic-style illustrations based on the AI-generated image prompts for each panel.
- The backend receives AI responses, formats the text and images appropriately, and passes them to the frontend templates for preview and export.

All AI model interactions happen dynamically during comic generation, ensuring that every comic is fully unique and customized based on user input.

Activity 1.3 Set up the development Environment:

Install Python & Pip:Ensure that **Python** is installed on your system along with **pip**, which will be used to manage the necessary project dependencies.

Create Virtual Environment:This ensures that project-specific packages do not conflict with system-wide installations.

```
python -m venv comiccraft-env

comiccraft-env\Scripts\activate   # For Windows
```

Install Required Libraries:
pip install fastapi uvicorn jinja2 python-multipart google-generativeai diffusers transformers fpdf Pillow accelerate

Run Server:
uvicorn app.main:app --reload
Visit http://127.0.0.1:8000 for the app and /docs for API testing.

Set Up Project Structure:
Create the initial project directory structure for ComicCraft:

```
comiccraft/
│
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   └── exporters.py
│
├── templates/
│   ├── index.html
│   ├── comic_preview.html
│   └── export_success.html
│
├── static/
│   ├── panels/        # Generated panel images
│   ├── exports/       # Exported comic PDFs
│   └── fonts/         # Fonts for PDF generation
│
├── .env               # Environment variables (API keys)
└── requirements.txt   # Python dependencies list
```

Milestone 2. Core functionalities Development